

AI Agent 学习手册（完整版 / 最新版）

适用路线: React + TypeScript + Node.js

说明: 本版把 8 个阶段全部按教学手册格式完整展开, 并统一更新到当前可访问的最新官方文档结构。
OpenAI 资料以 Developers 文档为准, LangChain / LangGraph 资料以当前 JS 官方文档为准。

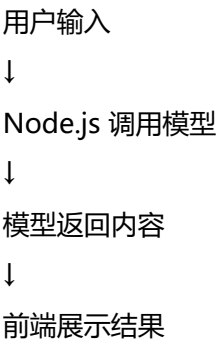
阶段	主题	建议时长	主要成果
第1阶段	LLM 基础	1周	基础 Chat Demo
第2阶段	Prompt Engineering	1周	Prompt 模板库
第3阶段	Tool Calling	2周	AI Tool Assistant
第4阶段	LangChain.js	2周	LangChain 版 Assistant
第5阶段	RAG / Agentic RAG	2周	企业知识库助手
第6阶段	LangGraph.js	2周	工作流 Agent
第7阶段	Subagents / Skills / MCP	2周	AI 团队助手
第8阶段	工程化部署	持续	可部署版本

第1阶段：LLM 基础（1周）

一句话目标：学会调用大模型 API，并理解一次请求、一次响应、一次多轮对话的完整结构。

为什么学习：这一阶段只做一件事：把模型调用彻底跑通。后面的 Prompt、Tool Calling、LangChain、RAG 都建立在这一层之上。

流程 / 关系



学习内容

Task	内容	目标
Task 1	理解 API 基础	知道模型、消息、输入输出分别是什么
Task 2	理解文本生成	学会发送文本提示并获得文本回答
Task 3	理解多轮对话	知道为什么需要维护对话状态
Task 4	理解流式输出	知道真实产品为什么更偏向 Streaming
Task 5	整理最小 Chat Demo	形成一个可持续迭代的项目底座

Task 1：理解 API 基础

为什么学习：如果基础输入输出都不清楚，后面所有框架看起来都会像黑盒。

推荐资料：OpenAI Quickstart。

资料	类型	建议阅读	优先级
OpenAI Quickstart	官方文档	Quickstart、API keys、第一次请求	★★★★★

[OpenAI Quickstart](#)

重点学习：API key、最小请求、最小响应、Node.js 调用方式。

暂时不要看：Agents SDK、MCP、Realtime。

练习任务：用 Node.js 发出第一次文本请求并打印结果。

Task 2：理解文本生成

为什么学习：Prompt 设计之前，先要知道文本输入和文本输出是怎么跑起来的。

推荐资料：Text generation。

资料	类型	建议阅读	优先级
Text generation	官方文档	文本生成输入输出结构	★★★★★

[OpenAI Text generation](#)

重点学习：输入消息、模型选择、返回内容结构。

暂时不要看：Structured outputs、Function calling。

练习任务：做一个只支持单轮问答的基础 API。

Task 3: 理解多轮对话

为什么学习：多轮对话是所有聊天类 Agent 的基础，后面 Tool Calling 回传结果也会用到同样的上下文思想。

推荐资料：Conversation state。

资料	类型	建议阅读	优先级
Conversation state	官方文档	多轮对话状态	★★★★★

[OpenAI Conversation state](#)

重点学习：历史消息、上下文延续、为什么模型会“记住”前文。

暂时不要看：工作流编排。

练习任务：让模型记住“我叫小白”，再回答“我叫什么”。

Task 4: 理解流式输出

为什么学习：流式输出会直接影响用户体验，后面做聊天产品基本离不开它。

推荐资料：Streaming。

资料	类型	建议阅读	优先级
Streaming	官方文档	流式输出基础	★★★★★

[OpenAI Streaming](#)

重点学习：流式返回、前端逐字渲染、体验差异。

暂时不要看：语音流、WebRTC。

练习任务：让聊天页面支持逐字显示回答。

Task 5: 整理最小 Chat Demo

为什么学习：后面最好不要每个阶段都重开项目，而是在同一个项目上持续升级。

推荐资料：复用前面所有资料。

资料	类型	建议阅读	优先级
OpenAI SDK	官方文档	Node.js SDK 基本调用	★★★★★

[OpenAI SDK](#)

重点学习：前后端分层、环境变量、错误处理、README。

练习任务：整理一个可持续迭代的 React + Node.js Chat Demo。

阶段产出

完成基础版 Chat Demo：React + TypeScript 前端，Node.js 后端，支持单轮问答、基本多轮上下文和流式输出。

本阶段常见误区

- 误区一：以为把页面做漂亮就代表学会了 API 调用，其实重点是请求结构和返回结构。
- 误区二：一上来就学框架，结果连最基础的模型调用过程都不清楚。
- 误区三：只看文档不动手，最后无法独立写出最小 Demo。

完成标准

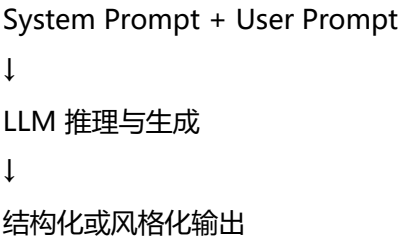
- 能独立调用一次 LLM API。

- 知道请求和响应的基本结构。
- 理解对话状态的作用。
- 完成基础聊天 Demo。

第2阶段：Prompt Engineering（1周）

一句话目标：学会让 LLM 按你的要求工作，并把 Prompt 当成可维护的程序资产。
为什么学习：Prompt 不是“会聊天”而已。对开发者来说，Prompt 是程序的一部分，是 Agent 的大脑。

流程 / 关系



学习内容

Task	内容	目标
Task 1	理解 Prompt	理解 Prompt 的作用和边界
Task 2	System Prompt	学会定义角色、风格和约束
Task 3	Few-shot Prompting	学会通过示例约束输出格式
Task 4	Chain of Thought	学会引导模型分步骤思考
Task 5	ReAct 思想	理解“思考 + 行动”的框架
Task 6	Prompt 模板库	沉淀可复用 Prompt 资产

Task 1：理解 Prompt

为什么学习：很多问题不是 Prompt 能解决的，先建立边界意识很重要。
推荐资料：Prompt engineering。

资料	类型	建议阅读	优先级
Prompt engineering	官方文档	Instructions、Role、Examples、Constraints	★★★★★

[OpenAI Prompt engineering](#)

重点学习：Instructions、clear constraints、role assignment。
暂时不要看：MCP、Agents SDK。
练习任务：总结 Prompt 能解决什么，不能解决什么。

Task 2：System Prompt

为什么学习：System Prompt 决定模型的行为方式，是后面 Agent 角色控制的基础。
推荐资料：Prompt engineering。

资料	类型	建议阅读	优先级
Prompt engineering	官方文档	Role、style、output constraints	★★★★★

[OpenAI Prompt engineering](#)

重点学习：Role、style、output constraints。
暂时不要看：工具编排。
练习任务：写 5 个不同角色的 System Prompt。

Task 3: Few-shot Prompting

为什么学习：复杂场景下，示例比口头描述更稳定。

推荐资料：Prompt Engineering Guide。

资料	类型	建议阅读	优先级
Prompt Engineering Guide	第三方参考	Few-shot 基本概念和示例	★★★★
Prompt generation	官方文档	辅助生成 Prompt 初稿	★★★

[Prompt Engineering Guide](#)

[OpenAI Prompt generation](#)

重点学习：示例驱动、一致格式、输入输出对齐。

暂时不要看：高级 Agent 模式。

练习任务：设计代码审查 Prompt，固定输出“问题/风险/建议/优化代码”。

Task 4: Chain of Thought

为什么学习：复杂问题如果不拆步骤，模型输出更容易发散。

推荐资料：Prompt Engineering Guide。

资料	类型	建议阅读	优先级
Prompt Engineering Guide	第三方参考	Chain-of-Thought 概念	★★★★

[Prompt Engineering Guide](#)

重点学习：step-by-step 分析、检查项、拆解顺序。

暂时不要看：推理模型深层参数。

练习任务：把“检查代码问题”改写成四步分析 Prompt。

Task 5: ReAct 思想

为什么学习：虽然这阶段还不写 Agent，但先理解 ReAct，后面 Tool Calling 会更顺。

推荐资料：Prompt Engineering Guide。

资料	类型	建议阅读	优先级
Prompt Engineering Guide	第三方参考	ReAct 思想与流程	★★★★

[Prompt Engineering Guide](#)

重点学习：Thought、Action、Observation、Answer。

暂时不要看：LangChain Agent 实现。

练习任务：用自然语言描述一个 ReAct 流程，不写代码。

Task 6: Prompt 模板库

为什么学习：这是后面做 Tool Calling、RAG、LangChain 时最值钱的复用成果。

推荐资料：复用前面所有资料。

资料	类型	建议阅读	优先级
Frontend prompting	官方文档	前端场景中的 Prompt 使用边界	★★★

[OpenAI Frontend prompting](#)

重点学习：可复用、可维护、按角色或任务拆分。

练习任务：建立 prompts/ 目录，至少写 5 到 10 个模板。

阶段产出

产出 prompts/ 模板库，至少包含 code_review、react_expert、node_expert、product_manager、interviewer 等模板。

本阶段常见误区

- 误区一：认为 Prompt 只是文案优化，不是工程资产。
- 误区二：把所有复杂问题都交给 Prompt，而不是配合 Tool、RAG、状态管理。
- 误区三：只会写一句长 Prompt，不会拆成可维护的模板。

完成标准

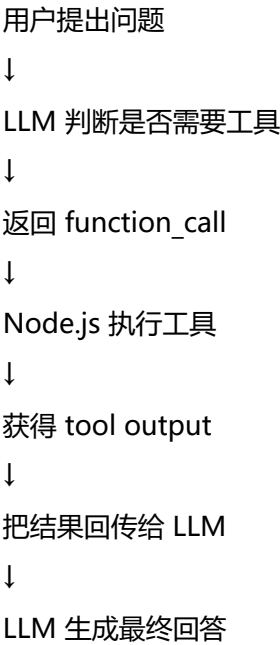
- 理解 Prompt 的作用和局限。
- 熟练编写 System Prompt。
- 会用 Few-shot Prompting。
- 理解 Chain of Thought 和 ReAct 的基本思想。
- 建立自己的 Prompt 模板库。

第3阶段：Tool Calling (2周)

一句话目标：学会让 LLM 调用外部工具，并完成第一个 AI Tool Assistant。

为什么学习：Prompt 只能让模型更会说，Tool Calling 才让模型真正开始“行动”。

流程 / 关系



学习内容

Task	内容	目标
Task 1	理解 Tool Calling	理解完整调用流程
Task 2	定义 Tools	学会定义工具名称、描述、用途
Task 3	JSON Schema	掌握工具参数的结构描述方式
Task 4	第一个 Tool Calling 闭环	完成一次完整工具调用
Task 5	Multiple Tools	让模型自动选择多个工具
Task 6	AI Tool Assistant	完成阶段项目

Task 1：理解 Tool Calling

为什么学习：没有 Tool Calling，LLM 永远只能聊天，不能真正行动。

推荐资料：Function calling。

资料	类型	建议阅读	优先级
Function calling	官方文档	tools、tool_choice、function_call、tool outputs	★★★★★

[OpenAI Function calling](#)

重点学习：tools、function_call、tool_choice、tool outputs。

暂时不要看：Agents SDK、MCP、Computer Use。

练习任务：画出完整工具调用流程图并解释谁执行工具。

Task 2：定义 Tools

为什么学习：工具描述不清楚，模型调用质量就会很差。

推荐资料：Function calling。

资料	类型	建议阅读	优先级
Function calling	官方文档	tool 定义、description、strict	★★★★★

[OpenAI Function calling](#)

重点学习：name、description、strict、参数设计。

暂时不要看：并行工具的复杂场景。

练习任务：定义 get_weather 工具及其描述。

Task 3: JSON Schema

为什么学习：模型不认识你的 TypeScript 类型，只认识你给它的 Schema。

推荐资料：JSON Schema。

资料	类型	建议阅读	优先级
JSON Schema	标准文档	object、properties、required、enum	★★★★★
Structured outputs	官方文档	模式约束和 JSON 输出	★★★★★

[JSON Schema](#)

[OpenAI Structured outputs](#)

重点学习：object、properties、required、enum、additionalProperties。

暂时不要看：复杂递归 schema。

练习任务：写出 weather、calculator、current_time 三个工具的 Schema。

Task 4: 第一个 Tool Calling 闭环

为什么学习：真正把模型判断、程序执行、结果回传串起来，Tool Calling 才算学会。

推荐资料：Function calling + SDK。

资料	类型	建议阅读	优先级
Function calling	官方文档	function_call_output 回传	★★★★★
OpenAI SDK	官方文档	Node.js 工具调用示例	★★★★★

[OpenAI Function calling](#)

[OpenAI SDK](#)

重点学习：arguments、call_id、function_call_output。

暂时不要看：Voice、Realtime。

练习任务：完成天气查询 Tool 的第一次调用。

Task 5: Multiple Tools

为什么学习：真实 Agent 不会只有一个工具，多工具是后面 LangChain 的基础。

推荐资料：Function calling。

资料	类型	建议阅读	优先级
Function calling	官方文档	tool_choice、parallel_tool_calls	★★★★★

[OpenAI Function calling](#)

重点学习：tool_choice、auto、required、parallel_tool_calls。

暂时不要看：LangChain Agent。

练习任务：加入 Calculator 和 Current Time 两个工具，支持自动选择。

Task 6: AI Tool Assistant

为什么学习：把第三阶段知识变成一个能演示、能写进简历的成品。

推荐资料：复用前面所有资料。

资料	类型	建议阅读	优先级
OpenAI SDK	官方文档	Node.js SDK 整体接入	★★★★

[OpenAI SDK](#)

重点学习：工具管理、错误处理、多轮对话、README。

练习任务：做一个天气、时间、计算器三类工具的 AI Tool Assistant。

阶段产出

完成 AI Tool Assistant：React + TypeScript 前端，Node.js 后端，支持 Weather / Calculator / Current Time 三类工具自动调用。

本阶段常见误区

- 误区一：以为 LLM 会自己调用天气 API，实际上执行的是你的程序。
- 误区二：把 Tool Result 直接返回给用户，而不是先回传给模型组织答案。
- 误区三：工具越多越好，实际上描述不清的工具越多，调用效果越差。

完成标准

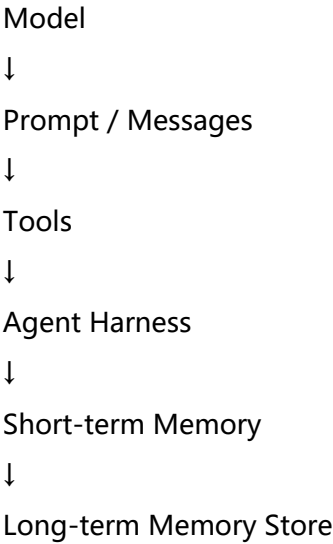
- 理解 Tool Calling 的完整流程。
- 会写 tools / function / parameters 定义。
- 会用 JSON Schema 描述参数。
- 能解析 function_call 并回传 function_call_output。
- 完成至少 3 个工具的自动选择。
- 完成 AI Tool Assistant 项目。

第4阶段：LangChain.js（2周）

一句话目标：把原生 API 经验迁移到框架层，学会用 LangChain.js 组织模型、Prompt、Tools、短期记忆、长期记忆和 Agent。

为什么学习：到这里你已经理解原理了，学习 LangChain.js 的重点不再是“它能做什么”，而是“它如何帮你更高效、更稳定地组织这些能力”。

流程 / 关系



学习内容

Task	内容	目标
Task 1	理解 LangChain 定位	理解 LangChain 是 Agent harness，不是模型本身
Task 2	模型与消息	掌握模型接入、消息组织方式
Task 3	Prompt 与 Tools	学会用 LangChain 方式声明 Prompt 和 Tool
Task 4	Agent Loop	理解 createAgent 的运行方式
Task 5	短期记忆与状态	让 Agent 能维护对话上下文
Task 6	长期记忆基础	理解并实现跨会话的用户记忆
Task 7	重构项目	把第三阶段项目升级为 LangChain.js 版本

Task 1：理解 LangChain 定位

为什么学习：很多人误以为学 LangChain 就是在学新模型，实际上它是组织模型和工具的框架。

推荐资料：Overview。

资料	类型	建议阅读	优先级
LangChain JS Overview	官方文档	Overview、Philosophy、Quickstart	★★★★★

[LangChain JS Overview](#)

重点学习：createAgent、harness、与 LangGraph 的关系。

暂时不要看：Deep Agents、Deployment。

练习任务：用自己的话解释 LangChain 与原生 OpenAI SDK 的关系。

Task 2：模型与消息

为什么学习：框架封装之后，消息和模型的组织方式仍然是核心。

推荐资料：Overview + Messages。

资料	类型	建议阅读	优先级
LangChain JS Overview	官方文档	模型接入与基本调用	★★★★★
LangChain Messages	官方文档	消息类型和对话结构	★★★★★

[LangChain JS Overview](#)

[LangChain Messages](#)

重点学习：messages、provider abstraction、调用入口。

暂时不要看：长时记忆。

练习任务：把你原来的 Chat Demo 接到 LangChain 模型接口上。

Task 3: Prompt with Tools

为什么学习：你已经会原生工具调用了，现在要学的是如何用框架更规范地组织。

推荐资料：Tools。

资料	类型	建议阅读	优先级
LangChain Tools	官方文档	tool 定义、schema、调用约束	★★★★★

[LangChain Tools](#)

重点学习：tool、schema、description、tool policies。

暂时不要看：复杂 middleware。

练习任务：把第三阶段的 weather / calculator / time 三个工具迁移到 LangChain.js。

Task 4: Agent Loop

为什么学习：这是 LangChain.js 的核心：让框架接管模型和工具之间的循环。

推荐资料：Agents。

资料	类型	建议阅读	优先级
LangChain Agents	官方文档	createAgent、Agent loop、tool usage	★★★★★

[LangChain Agents](#)

重点学习：createAgent、invoke、tool loop、agent output。

暂时不要看：多 Agent。

练习任务：做一个 LangChain 版 Tool Agent。

Task 5: 短期记忆与状态

为什么学习：没有状态，工具调用和多轮对话很难做得自然。

推荐资料：Short-term memory。

资料	类型	建议阅读	优先级
LangChain Short-term memory	官方文档	短期记忆与状态保持	★★★★★

[LangChain Short-term memory](#)

重点学习：对话状态、session 维持、上下文截断思路。

暂时不要看：跨会话长期记忆、语义检索和生产数据库。

练习任务：使用 checkpointer 和 thread_id 让 Agent 记住当前会话中的用户信息；换一个 thread_id 后验证该信息不会被读取。

Task 6: 长期记忆基础

为什么学习：短期记忆只属于单个 thread；长期记忆让同一用户在新会话中仍能取回偏好、身份信息和应用数据。

推荐资料：Long-term memory。

资料	类型	建议阅读	优先级
LangChain Long-term memory	官方文档	Store、JSON memory、namespace、key、读写长期记忆	★★★★★
LangChain Memory overview	官方文档	短期记忆与长期记忆的边界	★★★★

[LangChain Long-term memory](#)

[LangChain Memory overview](#)

重点学习：Store、namespace、key、userId、跨 thread 读取；理解 thread_id 与 userId 的职责差异。

暂时不要看：语义搜索、Embedding、PostgresStore、RedisStore、自动记忆抽取；这些在 RAG、LangGraph 和工程化阶段深入。

练习任务：使用 InMemoryStore 保存“用户偏好深色主题”，随后更换 thread_id、保持相同 userId，验证 Agent 仍能读取该偏好；再更换 userId，验证记忆隔离。

Task 7: 重构项目

为什么学习：你需要得到一个真正能用于后续 RAG 的工程底座。

推荐资料：复用前面所有资料。

资料	类型	建议阅读	优先级
LangChain Structured output	官方文档	结构化输出能力	★★★
LangChain JS Overview	官方文档	整体项目组织思路	★★★★

[LangChain Structured output](#)

[LangChain JS Overview](#)

重点学习：项目结构、服务层封装、README、测试。

练习任务：完成 LangChain.js 版 AI Assistant。

阶段产出

完成 LangChain.js 版 AI Assistant，保留第三阶段的工具能力，并加入短期会话状态与最小长期记忆：同一用户跨会话可恢复偏好，不同用户之间记忆隔离。

本阶段常见误区

- 误区一：一学框架就把原理忘了，结果出了问题完全不知道怎么查。
- 误区二：把 LangChain 当成“万能黑盒”，不理解 tool loop 背后的执行逻辑。
- 误区三：直接跳到复杂 Agent，不先把 createAgent 和 tools 打通。
- 误区四：把聊天记录持久化当成长期记忆；长期记忆需要跨 thread、按 userId 隔离并按需取回。

完成标准

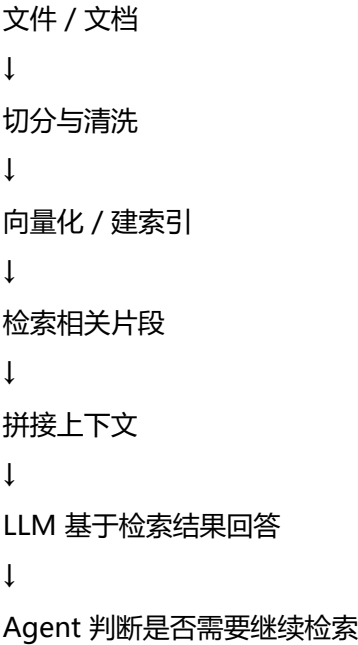
- 理解 LangChain.js 的定位和价值。
- 会用 createAgent 组织模型和工具。
- 能把原生工具迁移到 LangChain.js。
- 能维护短期上下文状态。
- 能解释短期记忆和长期记忆的区别。
- 能用 Store、namespace 和 key 保存并跨会话读取用户记忆。
- 能验证不同 userId 之间的记忆隔离。
- 完成 LangChain.js 版 AI Assistant。

第5阶段：RAG 与 Agentic RAG（2周）

一句话目标：学会把外部知识库接入 LLM，先完成可控的 2-step RAG，再理解由 Agent 决定何时检索的 Agentic RAG。

为什么学习：企业里最常见的 Agent 需求之一，就是让模型基于公司自己的文档和知识回答问题，而不是只靠通用知识。

流程 / 关系



学习内容

Task	内容	目标
Task 1	理解 RAG	理解检索增强生成的基本流程
Task 2	文档切分与清洗	学会把文档处理成适合检索的片段
Task 3	向量化与索引	理解 embeddings 在 RAG 中的作用
Task 4	检索器与问答链路	让模型基于检索结果回答问题
Task 5	回答质量与引用意识	提高 RAG 的可信度
Task 6	企业知识库助手	完成阶段项目
Task 7	Agentic RAG 入门	让 Agent 自己决定何时检索、是否重写问题以及何时回答

Task 1：理解 RAG

为什么学习：如果不理解 RAG 的目标，很容易把它误当成“上传文件给模型”这么简单。

推荐资料：LangChain Retrieval。

资料	类型	建议阅读	优先级
LangChain Retrieval	官方文档	Retrieval 概念、检索链路	★★★★★

[LangChain Retrieval](#)

重点学习：retrieval、retriever、上下文拼接、回答基于证据。

暂时不要看：复杂排序和多索引策略。

练习任务：画出完整 RAG 流程图。

Task 2: 文档切分与清洗

为什么学习：文档切得不合理，后面检索再强也很难返回有效答案。

推荐资料：LangChain Retrieval。

资料	类型	建议阅读	优先级
LangChain Retrieval	官方文档	chunking、metadata、文本清洗	★★★★★
OpenAI File inputs	官方文档	文件输入思路	★★★

[LangChain Retrieval](#)

[OpenAI File inputs](#)

重点学习：chunking、metadata、文本清洗。

暂时不要看：复杂 OCR 流程。

练习任务：拿一份 Markdown 或 PDF 文档做切分实验。

Task 3: 向量化与索引

为什么学习：向量化是把文本变成可检索表达的关键一步。

推荐资料：OpenAI Embeddings。

资料	类型	建议阅读	优先级
OpenAI Embeddings	官方文档	Embedding 基础和用途	★★★★★

[OpenAI Embeddings](#)

重点学习：embeddings 作用、索引思路、相似度检索概念。

暂时不要看：底层数学细节。

练习任务：把文档片段做向量化并建立简单索引。

Task 4: 检索器与问答链路

为什么学习：真正的 RAG 不是搜出来就结束，而是要把结果高质量地喂给模型。

推荐资料：LangChain Retrieval。

资料	类型	建议阅读	优先级
LangChain Retrieval	官方文档	retriever、context packing、grounded answer	★★★★★

[LangChain Retrieval](#)

重点学习：retriever、context packing、grounded answer。

暂时不要看：复杂 reranking。

练习任务：完成第一个基于文档的问答接口。

Task 5: 回答质量与引用意识

为什么学习：企业场景不只看能答，还看答案是否有依据、能否回溯。

推荐资料：Structured outputs（可选）+ 自己的项目文档。

资料	类型	建议阅读	优先级
OpenAI Structured outputs	官方文档	固定结构输出	★★★

[OpenAI Structured outputs](#)

重点学习：基于检索证据回答、引用片段、减少幻觉。

暂时不要看：复杂评测框架。

练习任务：让回答带上命中的文档片段或来源。

Task 6: 企业知识库助手

为什么学习：RAG 是企业中最常见、最有简历价值的 Agent 项目之一。

推荐资料：复用前面所有资料。

资料	类型	建议阅读	优先级
LangChain Retrieval	官方文档	整体 RAG 组装思路	★★★★★

[LangChain Retrieval](#)

重点学习：文件上传、索引更新、检索问答、项目说明。

练习任务：完成一个可上传文档并问答的知识库助手。

Task 7: Agentic RAG 入门

为什么学习：传统 2-step RAG 每次固定检索；Agentic RAG 把检索器变成工具，适合需要多次搜索或判断检索结果质量的复杂问题。

推荐资料：LangChain Retrieval + LangGraph Agentic RAG。

资料	类型	建议阅读	优先级
LangChain Retrieval	官方文档	RAG architectures: 2-step、Agentic、Hybrid	★★★★★
LangGraph Agentic RAG	官方教程	retriever tool、grade documents、rewrite question、generate	★★★★

[LangChain Retrieval](#)

[LangGraph Agentic RAG](#)

重点学习：2-step RAG 与 Agentic RAG 的边界；retriever tool；是否检索；相关性判断；查询改写。

暂时不要看：复杂多 Agent 检索、MCP、Skills、Subagents；这些放到第7阶段。

练习任务：给知识库助手增加检索工具：普通寒暄直接回答，知识库问题调用检索；记录每次是否检索以及命中文档。

阶段产出

完成企业知识库助手：支持上传文档、建立索引、检索相关片段、来源展示，并增加最小 Agentic RAG 模式，让 Agent 判断是否调用检索工具。

本阶段常见误区

- 误区一：以为 RAG 就是“把整个文档发给模型”。
- 误区二：只关注向量库，不关注切分、上下文拼接和答案约束。
- 误区三：做出能回答的 Demo，却无法说明回答依据来自哪里。
- 误区四：所有 RAG 都使用 Agentic RAG，增加不必要的调用、延迟和不确定性。

完成标准

- 理解 RAG 的完整流程。
- 会做文档切分与基础清洗。
- 理解 embeddings 在检索中的作用。
- 完成检索问答接口。
- 能解释 2-step RAG、Agentic RAG 和 Hybrid RAG 的区别。

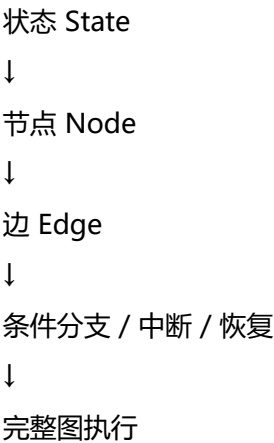
- 能把 retriever 封装成 Tool, 并让 Agent 判断是否调用。
- 完成企业知识库助手项目。

第6阶段：LangGraph.js (2周)

一句话目标：学会用 LangGraph.js 组织状态、节点、边和中断机制，完成复杂工作流 Agent。

为什么学习：当任务开始变复杂，只靠一个 Agent loop 往往不够，这时就要进入更底层、更可控的工作流编排。

流程 / 关系



学习内容

Task	内容	目标
Task 1	理解 LangGraph 定位	理解它与 LangChain 的差别
Task 2	State / Node / Edge	掌握图执行的三大核心概念
Task 3	条件分支与工作流	学会构建多步骤决策流程
Task 4	持久化与恢复	理解长流程 Agent 为什么要能恢复状态
Task 5	中断与人工介入	理解 human-in-the-loop 机制
Task 6	工作流 Agent 项目	完成阶段项目

Task 1：理解 LangGraph 定位

为什么学习：LangChain 更像 Agent harness，LangGraph 更像底层编排 runtime，这个定位一定要先分清。

推荐资料：LangGraph Overview。

资料	类型	建议阅读	优先级
LangGraph Overview	官方文档	Overview、Thinking in LangGraph、Workflows + agents	★★★★★

[LangGraph Overview](#)

重点学习：状态化工作流、编排、持久化、human-in-the-loop。

暂时不要看：部署细节。

练习任务：用自己的话比较 LangChain 和 LangGraph。

Task 2：State / Node / Edge

为什么学习：这是 LangGraph 的基本语法，如果不理解，后面所有图都是拼图。

推荐资料：Quickstart。

资料	类型	建议阅读	优先级
LangGraph Quickstart	官方文档	Graph API、Functional API	★★★★★

[LangGraph Quickstart](#)

重点学习：State、addNode、addEdge、START、END。

暂时不要看：高级运行时细节。

练习任务：做一个最小 hello world graph。

Task 3: 条件分支与 workflows

为什么学习：复杂 Agent 的关键不只是多个节点，而是不同路径如何切换。

推荐资料：Thinking in LangGraph + Workflows + agents。

资料	类型	建议阅读	优先级
Thinking in LangGraph	官方文档	条件分支和建模思路	★★★★★
Workflows + agents	官方文档	工作流与 Agent 结合	★★★★★

[Thinking in LangGraph](#)

[LangGraph Workflows + agents](#)

重点学习：条件分支、节点职责划分、流程控制。

暂时不要看：多人协作功能。

练习任务：做一个“分析 → 调工具 → 审核 → 输出”的图。

Task 4: 持久化与恢复

为什么学习：真实企业工作流往往不是一次调用就结束，持久化很关键。

推荐资料：Persistence。

资料	类型	建议阅读	优先级
LangGraph Persistence	官方文档	持久化与状态恢复	★★★★★

[LangGraph Persistence](#)

重点学习：checkpoints、断点恢复、长任务。

暂时不要看：生产部署平台细节。

练习任务：让图执行状态可以在中断后恢复。

Task 5: 中断与人工介入

为什么学习：很多企业流程并不能完全自动化，中间要允许人来确认和干预。

推荐资料：Interrupts。

资料	类型	建议阅读	优先级
LangGraph Interrupts	官方文档	中断、人工确认、恢复执行	★★★★★

[LangGraph Interrupts](#)

重点学习：interrupt、暂停、人工确认、恢复执行。

暂时不要看：复杂前端集成。

练习任务：给某个节点加一个人工确认步骤。

Task 6: 工作流 Agent 项目

为什么学习：把 LangGraph 真正用在复杂任务拆解上，才能体现它相对于 LangChain 的价值。

推荐资料：复用前面所有资料。

资料	类型	建议阅读	优先级
LangGraph Workflows + agents	官方文档	整体工作流 Agent 设计思路	★★★★

[LangGraph Workflows + agents](#)

重点学习：状态设计、节点边设计、项目说明。

练习任务：完成一个多步骤工作流 Agent。

阶段产出

完成工作流

Agent：至少包含需求分析、工具调用、审核或人工确认、最终输出四类步骤，并具备基本状态流转能力。

本阶段常见误区

- 误区一：把 LangGraph 当成更复杂的 LangChain，而不是状态化编排框架。
- 误区二：节点职责划分不清，导致图越画越乱。
- 误区三：只会画流程图，不会真正落地状态和恢复机制。

完成标准

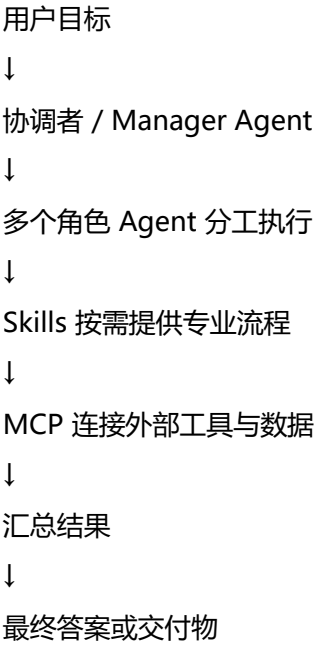
- 理解 LangGraph 的定位和优势。
- 掌握 State / Node / Edge 基本用法。
- 能做条件分支与多步骤流程。
- 理解持久化和中断恢复。
- 完成工作流 Agent 项目。

第7阶段：Agent 架构进阶（2周）

一句话目标：系统学习 Multi-Agent、Subagents、Skills 和 MCP，完成一个能够委派任务、按需加载能力并连接外部工具的 AI 团队助手。

为什么学习：当单个 Agent 的职责开始过大时，把任务拆给多个角色协作，通常比一个超级 Agent 更清晰、更容易维护。

流程 / 关系



学习内容

Task	内容	目标
Task 1	理解 Multi-Agent	理解为什么要拆成多个 Agent
Task 2	角色设计	学会定义多个 Agent 的职责边界
Task 3	Subagents 子代理	学会把复杂子任务委派给拥有独立上下文的子代理
Task 4	Agent Skills	学会把可复用 workflow 和领域知识封装为按需加载的 Skill
Task 5	MCP 工具协议	理解并实现 Agent 与外部工具服务的标准化连接
Task 6	Agent 间协作	理解多个 Agent 如何传递任务和结果
Task 7	冲突与回退	理解多个 Agent 结果不一致时怎么办
Task 8	AI 团队助手	完成阶段项目

Task 1：理解 Multi-Agent

为什么学习：不是所有任务都适合多 Agent，先理解边界才能避免过度设计。

推荐资料：LangChain Multi-agent + LangGraph Workflows + agents。

资料	类型	建议阅读	优先级
LangChain Multi-agent	官方文档	适用边界与主要模式	★★★★★
LangGraph Workflows + agents	官方文档	图式编排下的多 Agent 思路	★★★★★

[LangChain Multi-agent](#)

[LangGraph Workflows + agents](#)

重点学习：角色分工、任务拆解、协调与汇总。

暂时不要看：复杂企业协同平台。

练习任务：举 3 个适合多 Agent、3 个不适合多 Agent 的场景。

Task 2: 角色设计

为什么学习：多 Agent 成败常常不在数量，而在角色是否清晰。

推荐资料：LangGraph Workflows + agents。

资料	类型	建议阅读	优先级
LangGraph Workflows + agents	官方文档	角色边界和协作方式	★★★★★

[LangGraph Workflows + agents](#)

重点学习：Manager、Researcher、Developer、Reviewer 等角色边界。

暂时不要看：过细粒度角色拆分。

练习任务：为你的项目设计 3 到 4 个 Agent 角色。

Task 3: Subagents 子代理

为什么学习：子代理不仅是另一个角色，更重要的是隔离大量中间过程，避免主 Agent 上下文膨胀，并允许不同任务使用不同指令和工具。

推荐资料：Deep Agents Subagents。

资料	类型	建议阅读	优先级
Deep Agents Subagents	官方文档	使用边界、custom subagents、context management、tools	★★★★★

[Deep Agents Subagents](#)

重点学习：委派时机、context isolation、name、description、system prompt、最小工具权限、结果汇总。

暂时不要看：异步长任务、远程沙箱、大规模并行调度。

练习任务：创建 research 和 reviewer 两个子代理；主 Agent 只接收结构化结论，不接收子代理的全部中间消息。

Task 4: Agent Skills

为什么学习：Skill 适合承载较长、可复用、任务相关的说明和资源，避免把所有知识永久塞进 system prompt。

推荐资料：Deep Agents Skills。

资料	类型	建议阅读	优先级
Deep Agents Skills	官方文档	SKILL.md、目录结构、progressive disclosure、skills for subagents	★★★★★

[Deep Agents Skills](#)

重点学习：name、description、SKILL.md、附加脚本和资料、渐进式披露、主代理与子代理的 Skill 隔离。

暂时不要看：Interpreter Skills、复杂 Skill 市场和自动安装。

练习任务：创建一个 `rag-quality-review` Skill，包含检索质量检查步骤、引用检查模板和验收清单，并只分配给 reviewer 子代理。

Task 5: MCP 工具协议

为什么学习：普通 Tool 往往直接写在应用代码中；MCP 将工具和上下文通过标准协议暴露，使不同 Agent 客户端可以复用同一个工具服务。

推荐资料：LangChain MCP + Model Context Protocol。

资料	类型	建议阅读	优先级
LangChain MCP	官方文档	MultiServerMCPClient、getTools、custom server、stdio、HTTP	★★★★★
Model Context Protocol	协议文档	Architecture、Tools、Transports	★★★★★

[LangChain MCP](#)

[Model Context Protocol](#)

重点学习：MCP client/server、tool discovery、stdio 与 HTTP、鉴权意识、把 MCP tools 交给 createAgent。

暂时不要看：复杂 OAuth、多租户 MCP 网关、公开部署。

练习任务：用 TypeScript 创建一个只读知识库 MCP Server，再使用 '@langchain/mcp-adapters' 加载工具并交给 Agent 调用。

Task 6: Agent 间协作

为什么学习：多 Agent 的核心不是多个 prompt，而是协作机制。

推荐资料：LangGraph Workflows + agents。

资料	类型	建议阅读	优先级
LangGraph Workflows + agents	官方文档	任务分发、结果汇总、上下文共享	★★★★★

[LangGraph Workflows + agents](#)

重点学习：任务分发、结果汇总、上下文共享。

暂时不要看：分布式系统复杂细节。

练习任务：让 Manager Agent 把需求分配给两个子 Agent 再汇总。

Task 7: 冲突与回退

为什么学习：多 Agent 最大的问题不是能不能跑，而是结果冲突时如何处理。

推荐资料：复习 LangGraph 的分支与审核思想。

资料	类型	建议阅读	优先级
LangGraph Interrupts	官方文档	人工确认与中断恢复	★★★★★

[LangGraph Interrupts](#)

重点学习：review、fallback、人工确认。

暂时不要看：复杂投票机制。

练习任务：设计一个简单的冲突处理策略。

Task 8: AI 团队助手

为什么学习：多 Agent 最适合做“虚拟团队”类场景，便于展示和写简历。

推荐资料：复用前面所有资料。

资料	类型	建议阅读	优先级
LangChain Multi-agent	官方文档	模式选择与项目组合	★★★★★

[LangChain Multi-agent](#)

重点学习：角色协作、流程说明、项目边界。

练习任务：做一个 PM / Dev / Reviewer 协作的 AI 团队助手。

阶段产出

完成 AI 团队助手：包含 Manager、research 子代理和 reviewer 子代理；reviewer 按需加载 RAG 质量 Skill；Agent 通过 MCP 连接只读知识库工具，并完成拆分、委派、执行和汇总。

本阶段常见误区

- 误区一：觉得 Agent 越多越高级，结果系统变复杂、收益很小。
- 误区二：多个 Agent 职责重叠，最后谁都能做一点、谁都做不清。
- 误区三：只做角色名，没有真正的协作和汇总机制。
- 误区四：把 Subagent、Skill、Tool 和 MCP 当成同一个概念。
- 误区五：给每个子代理全部工具和全部 Skills，失去上下文隔离和最小权限的意义。

完成标准

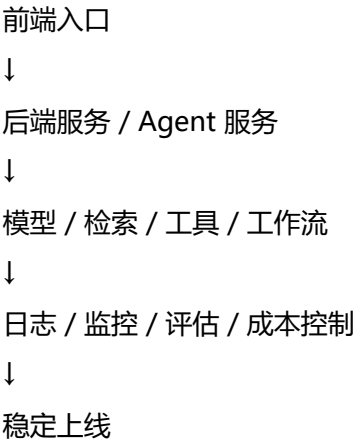
- 理解多 Agent 适用边界。
- 会设计清晰的角色职责。
- 能解释普通 Multi-Agent 与 Subagents 的区别。
- 能创建一个可复用 Skill，并控制主代理与子代理的 Skill 访问范围。
- 能创建或连接 MCP Server，并把 MCP tools 提供给 LangChain Agent。
- 能实现基本协作和结果汇总。
- 考虑冲突与回退机制。
- 完成 AI 团队助手项目。

第8阶段：工程化部署（持续）

一句话目标：把 Agent 做成真正能上线、能维护、能观察、能控制成本的工程系统。

为什么学习：企业真正关心的不是“能不能跑 Demo”，而是“能不能稳定上线、能不能排查问题、能不能控制成本”。

流程 / 关系



学习内容

Task	内容	目标
Task 1	理解工程化目标	知道上线系统和学习 Demo 的差异
Task 2	错误处理与重试	让系统在失败时可恢复
Task 3	限流与成本控制	理解线上系统的容量和成本约束
Task 4	可观测性与追踪	让系统出了问题时能定位
Task 5	部署与交付	形成一个可演示、可部署版本

Task 1：理解工程化目标

为什么学习：如果没有工程化意识，前面做出来的项目很难真正落地。

推荐资料：Production best practices。

资料	类型	建议阅读	优先级
Production best practices	官方文档	上线前的通用最佳实践	★★★★★
Deployment checklist	官方文档	部署前检查项	★★★★★

[OpenAI Production best practices](#)

[OpenAI Deployment checklist](#)

重点学习：稳定性、可观测性、安全性、成本。

暂时不要看：过早优化云架构细节。

练习任务：总结你的 Demo 距离“可上线”还差哪些点。

Task 2：错误处理与重试

为什么学习：模型调用、工具调用、检索调用都可能失败，错误处理是必须的。

推荐资料：Error codes + Production best practices。

资料	类型	建议阅读	优先级
OpenAI Error codes	官方文档	错误分类与处理	★★★★
Production best practices	官方文档	错误处理与回退思路	★★★★

[OpenAI Error codes](#)

[OpenAI Production best practices](#)

重点学习：超时、空响应、重试、回退策略。

暂时不要看：底层网络协议细节。

练习任务：为模型调用和工具调用加统一错误处理。

Task 3: 限流与成本控制

为什么学习：企业不会只问效果，还会问多少钱、能抗多少并发。

推荐资料：Rate limits。

资料	类型	建议阅读	优先级
OpenAI Rate limits	官方文档	限流和容量规划	★★★★

[OpenAI Rate limits](#)

重点学习：限流、吞吐、缓存、节流、请求策略。

暂时不要看：极端大规模架构。

练习任务：给项目加入基本限流和成本监控思路。

Task 4: 可观测性与追踪

为什么学习：Agent 的问题往往不是一个日志能看清，追踪链路特别重要。

推荐资料：LangSmith / Observability。

资料	类型	建议阅读	优先级
LangSmith Observability	官方文档	Tracing、调试、观察	★★★★

[LangSmith Observability](#)

重点学习：tracing、调试、节点级观察、输出比较。

暂时不要看：复杂告警平台整合。

练习任务：给你的 LangChain / LangGraph 项目接入基础 tracing。

Task 5: 部署与交付

为什么学习：这是你整个学习路线最终的成果展示阶段。

推荐资料：Deployment checklist。

资料	类型	建议阅读	优先级
Deployment checklist	官方文档	环境变量、构建、上线检查	★★★★★

[OpenAI Deployment checklist](#)

重点学习：环境变量、构建、上线检查、文档。

练习任务：整理一个可部署版本并写出部署说明。

阶段产出

完成一个可演示的线上版本，具备基本错误处理、日志、限流、部署说明和成本意识。

本阶段常见误区

- 误区一：Demo 能跑就以为可以上线。
- 误区二：只关注模型效果，不关注错误处理和监控。
- 误区三：没有日志和 tracing，系统一出问题就完全不知道卡在哪里。

完成标准

- 理解上线系统与 Demo 的区别。
- 有统一的错误处理和重试机制。
- 理解基本限流与成本控制。
- 接入基础可观测性。
- 完成可部署版本和部署文档。

最终简历项目总结

前面 8 个阶段不是彼此独立的 Demo，而是逐步沉淀为 3 个最适合写进简历的项目。
这一页的作用，就是把整条学习路线收束成最终可展示、可讲述、可包装的项目成果。

项目	核心能力	建议技术栈	简历亮点
AI Tool Assistant	Tool Calling、Prompt、基础多轮对话	React + TypeScript + Node.js + OpenAI SDK	体现你已经掌握 LLM 调用、工具定义、工具执行与结果回传的完整闭环。
企业知识库助手	RAG、Agentic RAG、文档问答、答案依据	LangChain.js + Embeddings + Retrieval Tools	体现你能接入企业私有知识，并让 Agent 按需检索、引用来源和控制回答依据。
工作流 / 多 Agent 助手	LangGraph.js、Subagents、Skills、MCP、人工审核	LangGraph.js + Deep Agents + MCP + Tracing	体现你能设计复杂 Agent 系统：任务委派、能力复用、外部工具接入、审批与可观测性。

项目包装建议

- 不要把 8 个阶段写成 8 个零散 Demo，应该收束为 2 到 3 个完整项目。
- 第 1 到 3 阶段的成果建议合并包装为 AI Tool Assistant。
- 第 4 到 5 阶段的成果建议合并包装为企业知识库助手。
- 第 6 到 8 阶段的成果建议合并包装为工作流 / 多 Agent 助手，并强调可观测性、审核机制和部署能力。
- 简历描述时优先写“解决了什么问题、系统如何工作、你负责了哪些关键模块”，不要只堆框架名。

最终总结

如果你把这 3 个项目真正做出来，并且每个项目都能讲清楚“目标、流程、技术选型、难点、效果”，
那你的简历定位就可以从“普通 Web 全栈开发”升级为“AI Agent 应用开发工程师 / LLM 全栈开发工程师”。